

Modern Practices for Training LLMs & Research with Agents

Duration: 2 days (4 half-day sessions, ~3h each) **Format:** Each session = ~1h30 presentation + ~1h interactive hands-on

Session 1 (Morning): Project Setup & Experiment Management

Objectives: Structure a reproducible DL research project with composable configs, live experiment tracking, and model/data management through the HuggingFace ecosystem.

Presentation (~1h30)

1. Why project structure matters

- argparse + print statements don't scale to teams or to your future self
- configs, scripts, logging, checkpoints, reproducibility

2. Hydra for configuration management

- Composable YAML configs. Config groups, overrides, multirun for sweeps.
- Integration with SLURM (one config, many jobs).
- Comparison with alternatives (argparse, OmegaConf standalone, jsonargparse).

3. Weights & Biases (W&B)

- Core concepts: runs, projects, sweeps, artifacts, reports.
- Logging metrics, hyperparameters, system stats, custom charts.
- Artifact versioning for datasets and checkpoints.
- Example: doing hyperparameter tuning with the hyperband algorithm

4. The HuggingFace ecosystem

- Transformers: AutoModel, AutoTokenizer, Trainer.
- Datasets: streaming, memory-mapping, custom data loading.
- The Hub: pushing/pulling models, datasets, tokenizers. Model cards.
- Accelerate: a bridge to distributed training.

Interactive session (~1h30)

Upgrade a baseline training script (fine-tuning a small LM on a text task):

1. Refactor hardcoded args into Hydra configs with config groups (model, data, training).
2. Add W&B logging (metrics, hyperparams, system monitoring).

3. Run a small Hydra multirun sweep tracked in W&B.
4. Push the best checkpoint to HuggingFace Hub.
5. All jobs submitted via SLURM.

Session 2: Distributed Training & Performance Optimization

Objectives: understanding GPU memory anatomy, scaling training from single-GPU to multi-node using DDP/FSDP/DeepSpeed, knowing how to profile and optimize throughput.

Presentation (~1h30)

1. GPU memory anatomy

- What fills VRAM: parameters, gradients, optimizer states, activations, fragmentation.
- Easy memory estimations for transformer models.
- Gradient checkpointing: trading compute for memory.

2. Data parallelism: DDP

- DistributedDataParallel internals: all-reduce, gradient bucketing, sync points.
- When DDP is enough (and when it isn't).
- torch.distributed basics: init_process_group, ranks, world_size.

3. Model parallelism & sharding

- FSDP (Fully Sharded Data Parallel): sharding strategies, mixed precision, wrapping policies.
- DeepSpeed ZeRO (Stages 1/2/3): what each stage shards, CPU/NVMe offloading.
- FSDP vs. DeepSpeed: practical decision criteria.

4. Attention & compute optimization

- FlashAttention: the memory bottleneck in standard attention, how tiling solves it, FlashAttention-2/3. Integration with HF and PyTorch.
- Mixed precision: bf16 vs. fp16 vs. fp32 — when each is appropriate.
- torch.compile: how it works, when it helps, current limitations.
- Data loading: num_workers, pin_memory, prefetching, streaming datasets.

5. SLURM for multi-node training

- Writing sbatch scripts for distributed jobs: srun, ntasks-per-node, NCCL env vars.

- Debugging distributed failures: NCCL timeouts, hung processes, log inspection.
- Practical tips: preemption handling, checkpoint-on-signal.

Interactive session (~1h30)

Starting from the Session 1 script:

1. Scale to multi-GPU with DDP on a single node (2-4 GPUs).
2. Switch to FSDP or DeepSpeed ZeRO-2 and compare memory usage.
3. Write a multi-node SLURM script (2 nodes) and launch.
4. Integrate FlashAttention and measure throughput difference.
5. Profile with PyTorch Profiler: generate a trace, identify the bottleneck, fix it.

Session 3: Agentic Workflows for Research

Objectives: what AI agents are beyond the hype, using Claude Code and OpenAI tools for common research tasks, and knowing the critical pitfalls to avoid.

Presentation (~1h30)

1. Agents demystified

- What agents actually are: tool use, planning loops, context management.
- The agent loop: observe → plan → act → reflect. How this differs from a single prompt.
- Context window as the fundamental constraint: what it means for research workflows.

2. The current tool landscape

- Claude Code, OpenAI Codex, ChatGPT Code Interpreter: terminal-native agentic coding. Extended thinking, file editing, bash integration...
- Other tools briefly: Cursor, GitHub Copilot
- Choosing the right tool for the right task.

3. Efficient research workflows with agents

- The “80/20” of agentic research: where agents save the most time.
 - Literature review and paper summarization.
 - Boilerplate code generation (training loops, data loaders, evaluation scripts).

- Debugging and error diagnosis.
- Data exploration and quick visualization.
- LaTeX formatting and figure generation.
- Workflow design: how to break a research task into agent-friendly subtasks.

4. Best practices & pitfalls

- Prompt engineering for agents: specificity, context-loading, incremental tasking.
- The “trust but verify” principle: when and how to check agent output.
- Hallucination risks: code that looks right but isn’t (subtle bugs, wrong API usage, fabricated references).
- Context window management: when to start fresh vs. continue.
- Anti-patterns: over-delegation, prompt chaining without review, using agents for tasks requiring domain judgment.
- No data is ever safe from an agent: be prepared to lose it all at all times

Session 4: Extending Agents — Skills, MCP, and Custom Pipelines

Objectives: extend agent capabilities with custom tools via MCP, create reusable skills, and design multi-step agent pipelines for research workflows.

Presentation (~1h)

1. Model Context Protocol (MCP)

- The problem: agents are isolated from your tools, data, and infrastructure.
- MCP architecture: servers, clients, transports, tool/resource/prompt primitives.
- How MCP lets agents talk to external systems: Slack, GitHub, databases, your SLURM cluster.
- The MCP ecosystem: existing servers, registries, community tools.

2. Skills: reusable agent capabilities

- What skills are: prompt + tool bundles that encode domain expertise.
- Skill design patterns: when to make a skill vs. a one-off prompt.
- Creating custom skills for recurring lab tasks (paper review, code review, experiment setup).
- Sharing skills within a research group.

3. Custom tool pipelines

- Chaining agent actions: “read paper → extract methods → generate config → submit SLURM job.”
- Claude Code hooks: pre/post-tool hooks, notification hooks.
- The Agent SDK: building custom agents with specific tool access.
- Simple scripting approaches: wrapping agent calls in bash/Python for automation.

4. Multi-agent patterns

- When to split work across specialized agents.
- Fan-out / fan-in: one orchestrator, multiple workers.
- Practical example: code-review agent + test-runner agent + documentation agent.

5. Responsible use

- Prompt injection awareness: what it is, why it matters when agents have tools.
- Data privacy: what gets sent to API providers, local vs. cloud models.
- When NOT to use agents: sensitive data, high-stakes decisions, regulatory contexts.
- Governance in a research group: shared guidelines, reproducibility concerns.

Interactive session (~2h)

Part 1 — Build an MCP server: Starting from a template, each team builds a small MCP server.

Suggested options:

- An arXiv search server (search papers, fetch abstracts).
- A SLURM cluster status server (query queue, check job status).
- A lab notebook server (read/write structured experiment notes).

Part 2 — Create a skill: Wrap the MCP server into a skill with a clear prompt template. Test it in Claude Code.

Part 3 — Automation: Teams combine their MCP server + skill into a small agent pipeline that automates a research workflow of their choice.